HappySite — Manual de Arquitetura

Sistema de administração de obras — Cerqueira Engenharia

> **Para que serve este documento:** descrever *como o sistema funciona por dentro* — onde está hospedado, como os dados fluem, onde fica cada coisa e por que foi feita daquele jeito. **Não é** um manual de operação (não ensina a usar as telas).
>
> **Público-alvo:** programadores, designers web, administradores de sistema e assistentes de IA que precisem dar manutenção ou evoluir o projeto.
>
> Última revisão: novembro de 2026.

Índice

1. Visão geral
2. Stack e decisões técnicas
3. Hospedagem e infraestrutura
4. Deploy automático
5. Mapa de pastas
6. Backend: anatomia
7. Modelo de dados
8. Autenticação e permissões
9. Frontend: anatomia
10. Domínios funcionais
11. Processos em segundo plano
12. Convenções e armadilhas
13. Dívida técnica e melhorias

---

1. Visão geral

O HappySite é um sistema web interno que administra o ciclo de trabalho de uma construtora: cadastro de obras, apontamento do que cada colaborador executou, cálculo do que cada um tem a receber, controle de EPI, ASO, agenda de compromissos e o financeiro de entrada.

O conceito central: a "célula"

Quase tudo no sistema gira em torno de uma ideia só. Uma obra é desenhada como um **prédio virtual**, e esse prédio é fatiado em **células** — cada célula é um pedaço mensurável da obra:

```
Cobertura
  Caixa d'água / Ático
7º Andar  →  Corredor | Elevador | Escada | Apto 701 | Apto 702 ...
6º Andar  →  Corredor | Elevador | Escada | Apto 601 | Apto 602 ...
...
Térreo    →  Corredor | Elevador | Escada | Vaga 1 | Vaga 2 ...
Laje Piso Térreo
Fundação - Etapa 1..N
```

Cada célula tem uma **chave textual** (`apto-b0-a3-2` , `escada-terreo` , `fundacao-1`) que é o identificador usado no banco inteiro. **Marcar uma célula** significa dizer: "*fulano executou este serviço neste pedaço da obra, neste mês*". Dessa marcação nasce todo o resto — a medição, o valor a pagar, os relatórios.

Fluxo macro

```
Cadastra obra (wizard: blocos, andares, aptos)
        ↓
Configura serviços da obra (alvenaria, reboco...) com preço unitário e equipe
        ↓
Marca células no desenho do prédio  ──→  grava em obra_servico_celulas (qtd + valor)
        ↓
Lança diárias e pagamentos antecipados (vale, FGTS, adiantamento...)
        ↓
Medição do mês = soma das células + diárias − descontos
        ↓
Fecha/paga a medição  →  status PAGO congela o histórico
```

Módulos

Módulo	O que resolve
Painel	Visão geral: avisos de ASO, experiência, resumo do mês
Obras	Cadastro da obra, desenho do prédio, serviços, marcação de células
Medição	Consolidação mensal do que cada pessoa tem a receber; fechamento e pagamento
Diárias	Lançamento de diárias por pessoa/mês, com rateio entre obras
Prestadores	Cadastro de colaboradores (CLT e PJ) e histórico
EPI	Estoque, retirada com assinatura digital e termo em PDF
ASO	Controle de validade do atestado de saúde ocupacional
Financeiro	Receitas (entradas), pagamentos, gastos e relatórios
Agenda	Compromissos com destinatários e lembretes push
Backup	Exportação/importação de todos os dados

---

2. Stack e decisões técnicas

Backend (`/server`)

Item	Escolha
Runtime	Node.js 22 LTS
Framework	Express 5
Banco	PostgreSQL 16 (driver `pg`)
Módulos	CommonJS (`require`)
Auth	`jsonwebtoken` + `bcryptjs`
Upload	`multer` (comprovantes em disco)
Planilhas	`xlsx` (importação/exportação)
Push	`web-push` (VAPID)
Drive	`googleapis` (backup externo)
Datas	`dayjs`

Frontend (`/client`)

Item	Escolha
Biblioteca	React 18 (apenas função + hooks, sem classes)
Build	Vite (variante Rolldown — exige Node 20.12+/22+)
Rotas	`react-router-dom`
HTTP	`axios` com interceptors
Estado	Context API (`AuthContext` , `ApuracaoContext`) — **sem Redux**
Estilo	CSS global (`index.css`) + estilos inline — **sem Tailwind**
PWA	Service Worker próprio (`sw-push.js`)

> **Regra ao evoluir:** não introduza bibliotecas novas sem necessidade real. O projeto é deliberadamente enxuto para ser mantido por uma pessoa só.

Três decisões que explicam muita coisa

1. Camada de compatibilidade SQL (`server/db/database.js`) O sistema nasceu em SQLite (`better-sqlite3`) e migrou para PostgreSQL. Para não reescrever centenas de queries, existe um adaptador que traduz na hora:

- placeholders `?` → `$1, $2, $3...`
- `INSERT` sem `RETURNING` ganha `RETURNING id` automaticamente (emula `lastInsertRowid`)
- expõe a API antiga: `db.get()` , `db.all()` , `db.run()` , `db.transaction()`

Por isso o código das rotas usa `?` mesmo sendo Postgres. **É intencional.**

2. Backend serve o frontend Não há servidor web separado para o React. O Express entrega a API em `/api/*` e também os arquivos estáticos de `client/dist` . Um único container, um único processo.

3. Migrações no boot Não há Prisma, Knex ou ferramenta de migration. O arquivo `database.js` roda um `migrate()` idempotente toda vez que o servidor sobe: cria tabelas com `CREATE TABLE IF NOT EXISTS` , adiciona colunas se não existirem, popula dados-semente. O servidor só aceita conexões depois que a promessa `db.pronto` resolve.

---

3. Hospedagem e infraestrutura

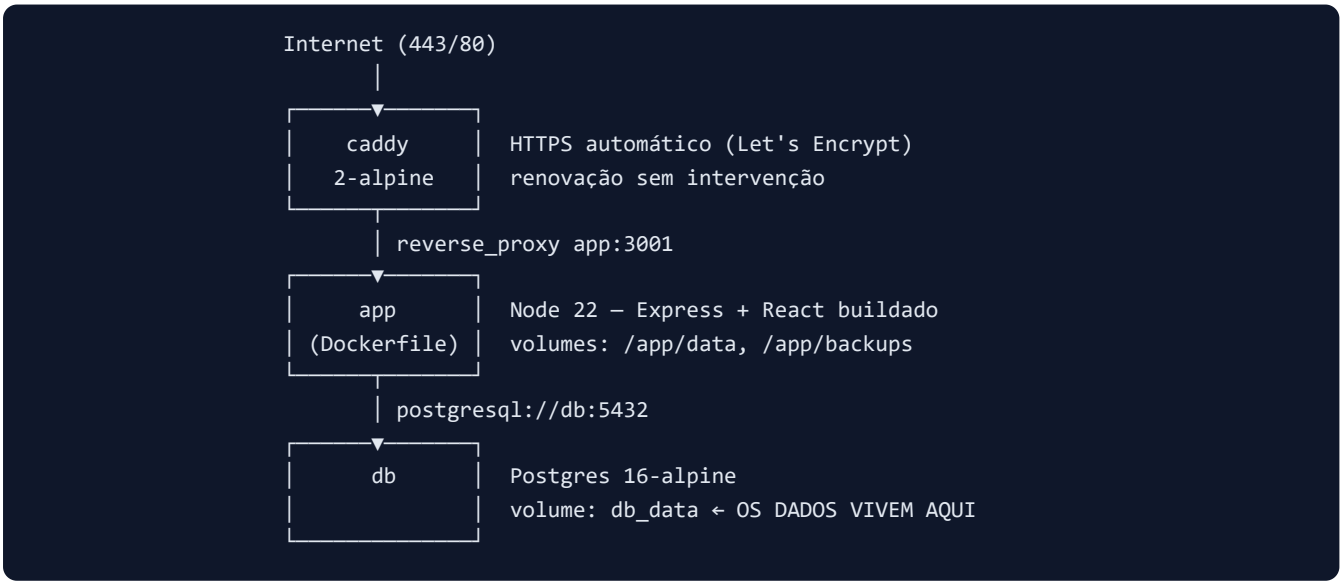
Onde está

Provedor	AWS Lightsail (VPS)
Região	us-east-1 (Virgínia) — **fuso diferente de Brasília**
IP	`3.228.110.68`
Domínio	`app.cerqueiraengenharia.com.br`
SO	Ubuntu
Caminho	`/opt/happysite`
RAM	2 GB (limitação relevante — ver adiante)
Acesso	`ssh -i ~/.ssh/deploy_happysite ubuntu@3.228.110.68`

Histórico: por que um VPS

A versão anterior usava Render (free) + Neon (free). Ambos hibernavam por inatividade, causando *cold start*: tela branca e espera longa no primeiro acesso do dia. A migração para um VPS único — com banco, aplicação e proxy no mesmo servidor — resolveu isso. Sobraram no código alguns vestígios dessa época (rota `/ping` de keep-alive, comentários sobre Render, `render.yaml`).

Os três containers



Definidos em `docker-compose.yml`. Todos com `restart: unless-stopped`. O `app` só sobe depois que o `db` passa no healthcheck (`pg_isready`).

Volumes (onde os dados persistem)

Volume	Conteúdo	Crítico?
`db_data`	**Todo o banco PostgreSQL**	🔴 Sim — perder isto é perder tudo
`app_data`	Comprovantes enviados (upload)	🟡 Sim
`app_backups`	Backups JSON automáticos	🟢 Recuperável
`caddy_data`	Certificados TLS	🟢 Recriável

> ⚠️ `docker compose down -v` **apaga os volumes**. Nunca use a flag `-v` em produção.

Dockerfile (build em dois estágios)

1. **Estágio build** — instala dependências do server (`--omit=dev`) e do client, roda

`vite build` gerando `client/dist` .

1. **Estágio final** — imagem limpa com apenas `server/` , `client/dist/` e `package.json` .

Comando: `node server/server.js` .

Node 22 é obrigatório: o Vite usado depende de `node:util.styleText` , ausente em versões antigas.

Variáveis de ambiente

Ficam no `.env` da **raiz do projeto no servidor** (`/opt/happysite/.env`), fora do Git.

Variável	Papel	Se faltar
`DB_PASSWORD`	Senha do Postgres	🔴 Nada funciona
`DATABASE_URL`	Montada pelo compose	—
`DB_SSL`	`false` no VPS (Postgres local)	Erro de conexão
`PORT`	`3001`	Usa 3001
`VAPID_PUBLIC_KEY` / `VAPID_PRIVATE_KEY`	Notificações push	Push desligado, resto normal
`VAPID_SUBJECT`	E-mail de contato VAPID	Usa padrão
`GOOGLE_SERVICE_ACCOUNT_JSON`	Backup no Drive	Backup só local

Módulos opcionais **degradam em silêncio** em vez de quebrar: sem VAPID, o push apenas não liga.

4. Deploy automático

`.github/workflows/deploy.yml` — dispara a cada push na `main` (ou manualmente em *Actions → Run workflow*). Leva de **5 a 10 minutos**.

```
push na main
   ↓
GitHub Actions (appleboy/ssh-action) conecta por SSH
   ↓
cd /opt/happysite
git fetch origin main && git reset --hard origin/main
   ↓
sudo docker image prune -f          ← libera RAM/disco ANTES do build
   ↓
sudo docker compose up -d --build   ← rebuild e restart
   ↓
sleep 15 && docker compose ps       ← confirma que subiu
```

Secrets necessários: `DEPLOY_KEY` (chave SSH privada), `DEPLOY_HOST`, `DEPLOY_USER`.

Detalhes que importam

- **`git reset --hard`, não `git pull`.** Pull falha silenciosamente se houver qualquer alteração

local no servidor. O reset garante paridade exata com o GitHub. Arquivos no `.gitignore` (como o `.env`) **não são afetados**.

- **`image prune` antes do build.** Com 2 GB de RAM, build + containers já travaram o servidor

por falta de memória.

- `concurrency: deploy-producao` impede dois deploys simultâneos corrompendo o build.
- `command_timeout: 20m` — o build leva de 3 a 6 minutos nesta instância.

⚠️ "Success" no GitHub ≠ aplicação no ar

O workflow pode concluir com sucesso e a aplicação estar quebrada (erro de runtime, container em loop de restart). **Sempre confirme de verdade:**

```
# 1. O workflow terminou?
Invoke-RestMethod 'https://api.github.com/repos/happySiteCerqueira/happysite/actions/runs?per_page=1'

# 2. O commit certo está no servidor e os containers de pé?
ssh -i ~/.ssh/deploy_happysite ubuntu@3.228.110.68 `
  "cd /opt/happysite && git rev-parse --short HEAD && sudo docker compose ps"

# 3. O site responde?
Invoke-WebRequest 'https://app.cerqueiraengenharia.com.br' -UseBasicParsing
```

Verificação pré-push

O build acontece no servidor. **Um erro de sintaxe JSX derruba o deploy** — já aconteceu. Se houver Node na máquina local, rode `npm --prefix client run build` antes do push. Sem Node, a checagem mínima é contar delimitadores:

```
$t = Get-Content 'client\src\pages\ObraDetalhe.jsx' -Raw
foreach ($p in @(@('{','}'),@('(',')'),@('[',']'))) {
  $a=([regex]::Matches($t,[regex]::Escape($p[0]))).Count
  $b=([regex]::Matches($t,[regex]::Escape($p[1]))).Count
  "$($p[0]) $a vs $($p[1]) $b -> $(if($a -eq $b){'OK'}else{'DESBALANCEADO'})"
}
```

Isso detecta desbalanceamento, mas **não** erros semânticos. É um paliativo, não uma garantia.

Comandos úteis no servidor

```
cd /opt/happysite
sudo docker compose ps                    # estado dos containers
sudo docker compose logs -f app           # logs em tempo real
sudo docker compose logs --tail 100 app   # últimas 100 linhas
sudo docker compose restart app           # reinicia só a aplicação
sudo docker compose exec db psql -U happysite -d happysite   # console SQL
```

---

5. Mapa de pastas

```
HappySite/
├── .github/workflows/deploy.yml   Deploy automático
├── client/                        Frontend React
│   ├── public/                    Ícones, logo, manifest
│   └── src/
│       ├── main.jsx               Entrada; registra SW e checa atualização
│       ├── App.jsx                Rotas e guarda de autenticação
│       ├── index.css              CSS global (.btn-*, .modal-*, .card)
│       ├── sw-push.js             Service Worker (push + cache PWA)
│       ├── api/api.js             Instância axios + interceptors
│       ├── context/
│       │   ├── AuthContext.jsx       Usuário logado, login/logout, temPermissao()
│       │   └── ApuracaoContext.jsx   Mês de apuração global
│       ├── components/
│       │   ├── Layout.jsx            Menu lateral, topo mobile, seletor de mês
│       │   ├── Modal.jsx           ★ Janela padrão (usar sempre esta)
│       │   ├── PredioDesenho.jsx     Desenho interativo do prédio
│       │   ├── ObraWizard.jsx        Assistente de criação de obra
│       │   ├── AssinaturaCanvas.jsx  Assinatura a dedo/mouse (EPI)
│       │   └── ErrorBoundary.jsx     Captura erros de render
│       ├── pages/                 Uma tela por módulo
│       └── utils/
│           ├── celulasPredio.js   ★ Gera as chaves das células
│           ├── medicaoExport.js      Exportação da medição
│           ├── termoEpiPdf.js        PDF do termo de EPI
│           └── notificacoes.js       Inscrição push no navegador
├── server/                        Backend Express
│   ├── server.js                  Bootstrap, rotas, estáticos, SPA fallback
│   ├── db/
│   │   ├── database.js          ★ Conexão, adaptador SQL e TODAS as migrações
│   │   └── seed.js                Dados iniciais
│   ├── routes/                    Um arquivo por recurso da API
│   └── utils/
│       ├── auth.js                JWT: gerarToken, autenticar, permitir
│       ├── permissaoModulo.js     Permissão por módulo (consulta o banco)
│       ├── recalculoAutomatico.js ★ Recálculo de valores lançados
│       ├── lembretes.js         ★ Agendador de notificações
│       ├── push.js                Envio Web Push
│       ├── aso.js                 Regras de vencimento do ASO
│       ├── celulasLabel.js        Chave da célula → rótulo legível (backend)
│       ├── backupAutomatico.js    Backup diário
│       ├── googleDrive.js         Upload para o Drive
│       ├── auditoria.js           Registro de ações
│       └── verificarRotas.js    ★ Trava: rota não registrada = não sobe
├── data/comprovantes/             Uploads (volume app_data)
├── backups/                       Backups JSON (volume app_backups)
├── docker-compose.yml             Orquestração
├── Dockerfile                     Build da aplicação
├── Caddyfile                      Proxy e HTTPS
├── DEPLOY_VPS.md                  Passo a passo da infraestrutura
└── ARQUITETURA_SISTEMA.md         Este documento
```

★ = arquivos centrais, leia primeiro ao assumir o projeto.

> **Nota:** `tools-cor-notebook/` e `ALTERAR DE COMPUTADOR/` são utilitários avulsos, sem relação com a aplicação.

Os maiores arquivos (atenção redobrada)

Arquivo	Linhas	Observação
`client/src/pages/ObraDetalhe.jsx`	~2000	Tela mais complexa; candidata a divisão
`server/routes/obras.js`	~1300	Serviços, células, grupos, quantidades
`client/src/pages/Financeiro.jsx`	~980	Quatro abas numa tela só
`client/src/pages/Agenda.jsx`	~870	Calendário + destinatários + lembretes
`server/db/database.js`	~800	Esquema e histórico de migrações

6. Backend: anatomia

Sequência de inicialização (`server/server.js`)

```
1. require('./db/database')      → dispara as migrações (promessa db.pronto)
2. verificarRotasRegistradas()   → se faltar app.use, process.exit(1)
3. Middlewares: cors, express.json (limite 10 MB)
4. GET /ping                     → keep-alive (sem auth, sem banco)
5. /comprovantes                 → arquivos enviados (estáticos)
6. app.use('/api/...')           → 17 routers
7. Estáticos do client/dist + fallback do SPA
8. await db.pronto → app.listen(3001)
9. iniciarBackupAutomatico() e iniciarAgendadorLembretes()
```

> O passo 8 é importante: o servidor **não aceita requisições antes das migrações terminarem**. Se a migração falhar, o processo sai com código 1 e o container reinicia.

A trava de rotas (`verificarRotas.js`)

Já aconteceu de um módulo inteiro (EPI) existir em `routes/` e **sumir do site** porque faltava o `app.use()` correspondente. Agora, se houver um arquivo em `server/routes/` sem registro no `server.js`, o servidor **recusa subir**. Falha de propósito, para não publicar versão quebrada.

Consequência prática: ao criar `server/routes/novo.js`, registre-o obrigatoriamente:

```
app.use('/api/novo', require('./routes/novo'));
```

Cache e o fallback do SPA

Dois problemas reais resolvidos aqui:

1. App preso em versão antiga. Arquivos com hash no nome (`index-AbC123.js`) podem ser cacheados indefinidamente. Mas `index.html` , `sw-push.js` , `sw.js` , `manifest.webmanifest` e `registerSW.js` recebem `Cache-Control: no-cache, no-store, must-revalidate` — sem isso, o app instalado no celular nunca percebia que havia versão nova.

2. Tela branca após deploy. O fallback do SPA só devolve `index.html` para rotas de *navegação* (sem extensão no último segmento). Pedidos a arquivos estáticos inexistentes recebem 404 de verdade. Antes, um bundle antigo recebia o HTML com status 200, o navegador tentava executar HTML como JavaScript, falhava em silêncio e o React nunca montava.

```
app.get(/^(?!\/api).*/, (req, res, next) => {
  const ultimoSegmento = req.path.split('/').pop() || '';
  if (ultimoSegmento.includes('.')) return next();  // 404 real
  res.setHeader('Cache-Control', 'no-cache, no-store, must-revalidate');
  res.sendFile(path.join(clientBuildPath, 'index.html'));
});
```

Rotas da API

Rota	Arquivo	Responsabilidade
`/api/auth`	`auth.js`	Login, troca de senha
`/api/usuarios`	`usuarios.js`	CRUD de usuários do sistema
`/api/colaboradores`	`colaboradores.js`	Cadastro de pessoas (CLT/PJ)
`/api/obras`	`obras.js`	Obras, serviços, células, grupos, quantidades
`/api/medicoes`	`medicoes.js`	Consolidação mensal, fechamento, comprovantes
`/api/diarias`	`diarias.js`	Diárias e rateio entre obras
`/api/pagamentos-antecipados`	`pagamentos-antecipados.js`	Vale, FGTS, adiantamento
`/api/prestadores`	`prestadores.js`	Visão de prestadores e histórico
`/api/epi`	`epi.js`	Estoque, retiradas, assinatura
`/api/aso`	`aso.js`	Vencimentos e histórico de ASO
`/api/financeiro`	`financeiro.js`	Receitas, importação de planilha
`/api/agenda`	`agenda.js`	Compromissos, destinatários, lembretes
`/api/push`	`push.js`	Inscrição de dispositivos
`/api/painel`	`painel.js`	Indicadores da tela inicial
`/api/permissoes`	`permissoes.js`	Matriz perfil × módulo
`/api/relatorios`	`relatorios.js`	Relatórios diversos
`/api/backup`	`backup.js`	Exportar/importar dados

Padrão de um arquivo de rota

```
const express = require('express');
const db = require('../db/database');
const { autenticar, permitir } = require('../utils/auth');
const { permissaoModulo } = require('../utils/permissaoModulo');
const { registrar } = require('../utils/auditoria');

const router = express.Router();

router.use(autenticar);                    // exige token
router.use(permissaoModulo('financeiro')); // exige permissão do módulo

router.get('/receitas', async (req, res) => {
  const linhas = await db.all('SELECT * FROM financeiro_receitas WHERE id = ?', id);
  res.json(linhas);
});

module.exports = router;
```

Observe: placeholders `?` (traduzidos pelo adaptador) e parâmetros **soltos**, não em array.

Transações

Use quando várias gravações precisam ser atômicas:

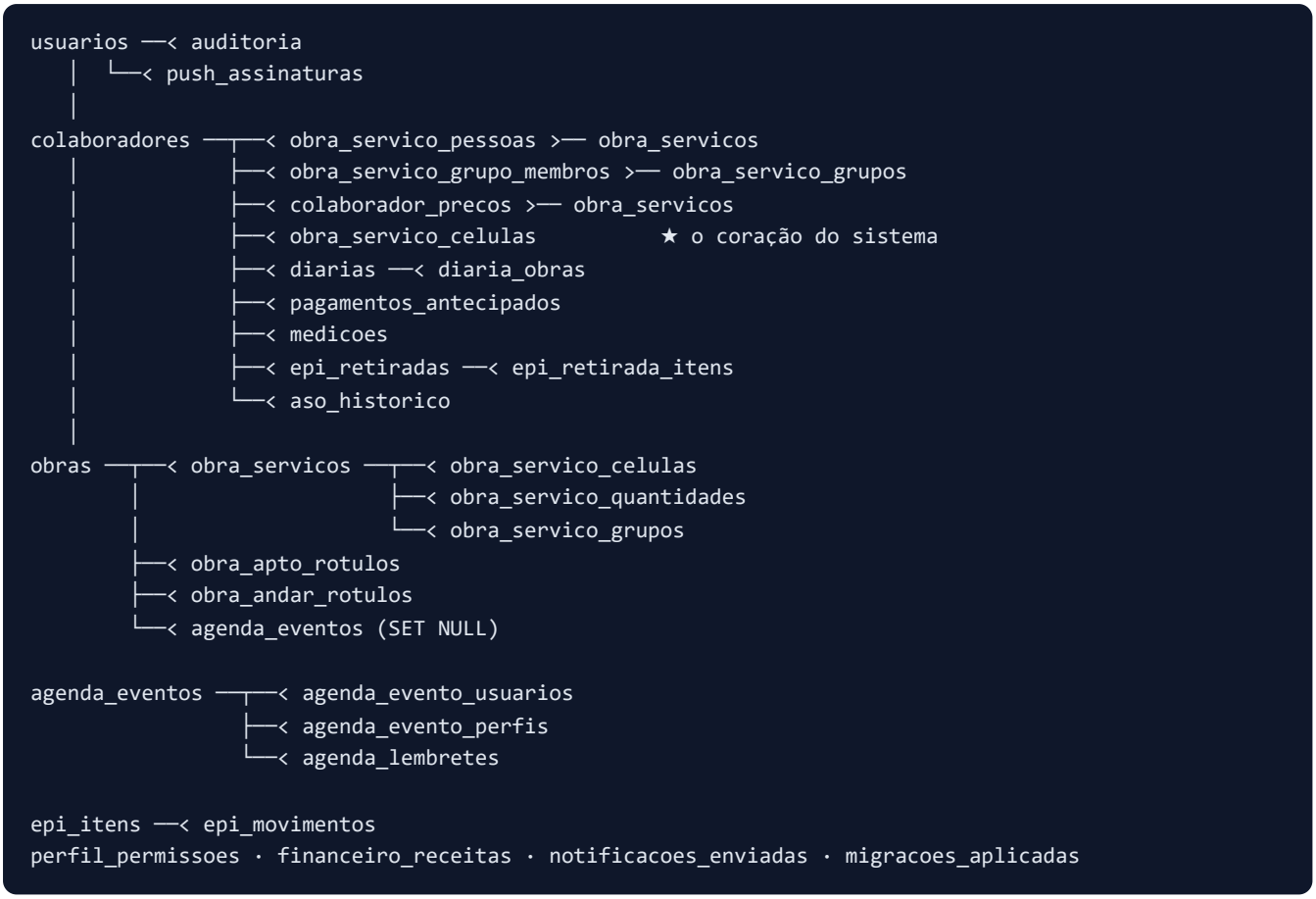
```
await db.transaction(async (trx) => {
  await trx.run('INSERT INTO ...');
  await trx.run('UPDATE ...');
  // erro aqui = ROLLBACK automático
});
```

---

7. Modelo de dados

PostgreSQL 16, **32 tabelas**. Todo o esquema está em `server/db/database.js`.

Diagrama de relacionamentos



`colaboradores` — as pessoas

Coluna	Observação
`tipo`	`'CPF'` (CLT) ou `'PJ'` (empreiteiro) — **distinção usada em várias regras**
`nome`, `documento`, `telefone`, `email`, `endereco`	Dados básicos
`funcao`	Cargo; em PJ usa-se `contato_responsavel`
`banco`, `agencia`, `conta`, `pix`	Dados de pagamento
`cor`	Cor da pessoa no desenho do prédio
`valor_diaria`	Base do cálculo de diárias
`ativo`	`1` / `0`
`data_admissao`	Base do período de experiência
`experiencia_status`	`EM_EXPERIENCIA` / `EFETIVADO` / `DISPENSADO`
`confirmado_45_dias`	Marco intermediário
`data_primeiro_aso`	Base do vencimento de ASO
`data_desligamento`	Carência de 35 dias nos lançamentos

> **Regra recorrente:** ASO e período de experiência valem **apenas para** `tipo = 'CPF'`. PJ não tem exame ocupacional nem experiência.

`obras` — a estrutura do prédio

A geometria fica em colunas **JSON serializado como TEXT** (herança do SQLite):

Coluna	Formato
`blocos_pavimentos`	`[{ qtd_andares, apto_por_andar }, ...]`
`itens_terreo`	`["Portaria", "Salão de festas", ...]`
`itens_cobertura`	`["Casa de máquinas", ...]`
`terreo_tipo`	`'estacionamento'` ou `'apartamento'`
`tem_transicao`, `tem_atico`, `tem_caixa_dagua`	Flags `1` / `0`
`fundacao_etapas`	Número de etapas

> ⚠️ São strings JSON, **não** colunas `jsonb`. Exigem `JSON.parse` / `stringify`. Migrar para `jsonb` seria uma melhoria real, mas exige alterar todas as leituras.

`obra_servicos` — serviço contratado numa obra

Coluna	Valores		
`modo_medicao`	`'apartamento'` \	`'pavimento'` \	`'frente_fundo'` — define quais células existem
`modo_execucao`	`'individual'` \	`'grupo'`	
`valor_unitario`	Preço padrão por unidade		
`unidade`	`'m²'`, `'un'` …		
`ordem`	Ordem de exibição		

`obra_servico_celulas` — ★ o coração do sistema

Cada linha = **uma pessoa executou um serviço numa célula, num mês**.

```
id, obra_servico_id, celula_key, colaborador_id, grupo_id,
mes_ciclo,        -- 'YYYY-MM'
quantidade,       -- parte DESTA pessoa (já rateada)
valor,            -- parte DESTA pessoa (já rateada)
criado_por, criado_em
UNIQUE(obra_servico_id, celula_key, mes_ciclo, colaborador_id)
```

Regra de ouro do rateio: se uma `(celula_key, mes_ciclo)` tem **mais de uma linha**, é execução compartilhada — vale tanto para grupo cadastrado quanto para equipe avulsa. Nesse caso `quantidade` e `valor` de cada linha são **a fração daquela pessoa**, não o total da célula. Para obter o total da célula, **some as linhas**.

```
const execucaoCompartilhada = !!linhas[0].grupo_id || linhas.length > 1;
const qtdGravadaTotal = linhas.reduce((acc, l) => acc + (Number(l.quantidade) || 0), 0);
```

Tabelas de apoio das obras

Tabela	Papel
`obra_servico_quantidades`	Metragem cadastrada por célula (a "verdade" do tamanho)
`obra_servico_pessoas`	Quem está alocado no serviço
`obra_servico_grupos`	Equipes fixas; `grupo_vinculo_id` sincroniza membros entre obras
`obra_servico_grupo_membros`	Integrantes do grupo
`colaborador_precos`	Preço unitário **específico** de uma pessoa (sobrepõe o do serviço)
`obra_apto_rotulos`	Renomear apartamento (ex: "302" → "Cobertura A")
`obra_andar_rotulos`	Renomear andar

Financeiro e pagamento

Tabela	Chave única	Conteúdo
`diarias`	`(colaborador_id, mes_ciclo)`	Quantidade, unitário usado, total
`diaria_obras`	`(colaborador_id, mes_ciclo, obra_id)`	Rateio das diárias por obra
`pagamentos_antecipados`	`(colaborador_id, mes_ciclo)`	`vale`, `fgts`, `taxa`, `pagto`, `vale_extra`, `adiantamento`, `vale_ex_rh`
`medicoes`	`(colaborador_id, mes_ciclo, obra_id)`	Bruto, descontos, líquido, `status`, comprovante
`financeiro_receitas`	—	Entradas: obra, serviço, bruto/líquido, `com_nota`, status

`medicoes.status`: `PENDENTE` → `APROVADO` → `PAGO`. **`PAGO` congela o histórico** — nenhum recálculo automático toca meses pagos.

`financeiro_receitas.servico` aceita apenas: `Pintura`, `Produção`, `Diárias`, `Reforma N`, `Reforma S`. Regra do líquido: `com_nota = true` → desconta 11 % (`bruto × 0,89`); `com_nota = false` → líquido igual ao bruto.

> Valores de `pagamentos_antecipados` podem ser **negativos** (saldo a favor da pessoa). Por isso as consultas usam `<> 0` e não `> 0` — filtrar só positivos fazia a pessoa sumir da Medição.

Agenda e notificações

Tabela	Papel
`agenda_eventos`	`data`, `data_fim` (período), `periodo` (`HORA` / `DIA_INTEIRO` / `MANHA` / `TARDE` / `NOITE`), `hora`, `obra_id`, `concluido`
`agenda_evento_usuarios`	Destinatários individuais
`agenda_evento_perfis`	Destinatários por perfil (ex: todo o RH)
`agenda_lembretes`	"avise X minutos antes" (vários por evento)
`push_assinaturas`	Um registro **por dispositivo** (`endpoint` único)
`notificacoes_enviadas`	Chave única do envio — impede duplicidade

A chave de `notificacoes_enviadas` segue um padrão legível: `agenda:12:1440` (evento 12, 24 h antes), `aso:7:2026-09-20:lembrete`, `exp:3:45:2026-10-01:dia`.

EPI, ASO e controle

Tabela	Papel
`epi_itens`	Catálogo com `ca`, `quantidade`, `estoque_minimo`
`epi_retiradas`	Retirada com `assinatura` (imagem base64)
`epi_retirada_itens`	Itens da retirada (descrição e CA **copiados** — histórico imutável)
`epi_movimentos`	`ENTRADA` / `SAIDA` do estoque
`aso_historico`	Cada renovação de ASO
`perfil_permissoes`	Matriz `(perfil, modulo) → permitido`
`auditoria`	Ação, entidade, detalhes, usuário
`migracoes_aplicadas`	Controle de migrações de execução única

Convenções do esquema

- `mes_ciclo` é sempre `TEXT` no formato `'YYYY-MM'`. É a dimensão temporal de quase tudo.
- **Booleanos como** `INTEGER` (`1` / `0`) — herança do SQLite. Exceção: `financeiro_receitas.com_nota`, que é `BOOLEAN` de verdade.
- **Enums via** `CHECK`, não tipos `ENUM` do Postgres. Para aceitar um valor novo é preciso derrubar e recriar a constraint (há exemplos no `database.js`).
- `ON DELETE CASCADE` nos filhos; `SET NULL` onde o histórico deve sobreviver

(apagar uma obra não apaga o compromisso da agenda, apenas o desvincula).

Como escrever uma migração

```
// 1) Coluna nova — sempre checando antes
if (!(await colunaExiste('tabela', 'coluna_nova'))) {
  await pool.query('ALTER TABLE tabela ADD COLUMN coluna_nova TEXT');
  await pool.query('UPDATE tabela SET coluna_nova = ... WHERE ...');  // backfill
}

// 2) Tabela nova
await pool.query(`CREATE TABLE IF NOT EXISTS nova (...);`);

// 3) Operação única e irrepetível
const ja = await db.get('SELECT 1 FROM migracoes_aplicadas WHERE nome = ?', 'minha_migracao');
if (!ja) {
  /* ... executa ... */
  await db.run('INSERT INTO migracoes_aplicadas (nome) VALUES (?)', 'minha_migracao');
}
```

> 🔴 **Armadilha histórica:** `constraintContem()` compara o **nome** da constraint, não o conteúdo. Usar isso como verificação derrubou o servidor num loop de `process.exit(1)`. Hoje a checagem correta usa `pg_get_constraintdef()`.

---

8. Autenticação e permissões

Fluxo de login

```
POST /api/auth/login { login, senha }
   ↓  bcrypt.compare contra senha_hash
   ↓  jwt.sign({ id, login, perfil, nome }, SECRET, { expiresIn: '12h' })
   ↓
Cliente guarda em localStorage: hs_token, hs_usuario
   ↓
Toda requisição: Authorization: Bearer <token>   (interceptor do axios)
   ↓
Resposta 401 → limpa storage e redireciona para /login
```

Senha inicial do admin: `admin` / `admin`, com `precisa_trocar_senha = 1` forçando a troca no primeiro acesso.

Os sete perfis

Perfil	Alcance
`ADM`	**Tudo.** Ignora qualquer verificação de permissão
`RH`	Pessoas, EPI, ASO, diárias, prestadores, financeiro parcial
`FINANCEIRO`	Medição, financeiro completo, diárias, prestadores
`ENGENHEIRO`	Obras, EPI, agenda
`MESTRE`	Obras, EPI, agenda
`ENCARREGADO`	**Somente EPI** — perfil restrito
`APONTADOR`	Obras, EPI, agenda

> O perfil `ENCARREGADO` substituiu o antigo `SUPERVISOR` (houve migração dos dados). Seu tratamento é especial em três lugares: menu lateral oculto (`Layout.jsx`), tela inicial redirecionada para `/epi` (`App.jsx` → `TelaInicial`) e Configurações reduzida a "Minha conta".

Duas camadas de autorização

1. `permitir(...perfis)` — verificação estática, codificada na rota:

```
router.get('/relatorio', permitir('ADM', 'FINANCEIRO'), handler);
// ADM sempre passa, independentemente da lista
```

2. `permissaoModulo('nome')` — verificação dinâmica, consultando `perfil_permissoes` :

```
router.use(permissaoModulo('financeiro'));
```

A segunda é a preferida: permite ao ADM reconfigurar acessos pela tela **Controle de Acesso** sem alterar código.

Módulos e sub-módulos

A tabela `perfil_permissoes` aceita granularidade com ponto:

```
obras · medicao · diarias · prestadores · epi · financeiro · aso · agenda
financeiro.receita · financeiro.pagamentos · financeiro.gastos
financeiro.relatorios · financeiro.resumo
prestadores.cadastro · epi.cadastrar
```

Os padrões iniciais são semeados em `database.js` . Submódulos novos são inseridos com `ON CONFLICT DO NOTHING` , preservando configurações já ajustadas manualmente.

No frontend

`AuthContext` expõe `temPermissao(...perfis)`, usada para esconder itens de menu e botões:

```
{temPermissao('FINANCEIRO', 'RH') && <NavLink to="/financeiro">💵 Financeiro</NavLink>}
```

> ⚠️ **Esconder no frontend não é segurança.** É conveniência visual. A proteção real está sempre no backend. Toda rota nova precisa do seu próprio middleware.

Pontos fracos conhecidos

Problema	Onde	Gravidade
`JWT_SECRET` fixo no código-fonte	`server/utils/auth.js`	🔴 Alta — deveria vir do `.env`
Token em `localStorage`	`api.js`	🟡 Vulnerável a XSS; cookie `httpOnly` seria melhor
Sem refresh token	—	🟢 Baixa — expira em 12 h e exige novo login

---

9. Frontend: anatomia

Inicialização

`main.jsx` monta a árvore de providers e registra o Service Worker:

```
<AuthProvider>
  <ApuracaoProvider>
    <BrowserRouter>
      <ErrorBoundary>
        <App />
```

Os dois contextos

`AuthContext` — usuário logado, `login()`, `logout()`, `temPermissao()`. Lê `hs_token` e `hs_usuario` do `localStorage` ao iniciar.

`ApuracaoContext` — o **mês de apuração global** (`'YYYY-MM'`), exibido no topo e usado por Medição, Diárias, Pagamentos Antecipados, histórico de Prestadores e marcação de células.

Detalhe deliberado: fica em **`sessionStorage`**, não `localStorage`. Ao reabrir o app ou dar F5, volta sempre ao mês vigente; durante a navegação, permanece onde o usuário deixou. Exceção: *Financeiro → Resumo*

tem período próprio.

Rotas

Caminho	Tela
`/login`, `/trocar-senha`	Públicas
`/`	`TelaInicial` → Painel (ou `/epi` se ENCARREGADO)
`/obras`, `/obras/:id`	Lista e detalhe da obra
`/medicao`, `/diarias`, `/prestadores`	Operação
`/epi`, `/aso`, `/financeiro`, `/agenda`	Módulos
`/configuracoes`	Conta, usuários, permissões, backup

O componente `<Privado>` protege tudo: sem usuário → `/login`; com `precisa_trocar_senha` → `/trocar-senha`.

Padrão de uma página

```
export default function MinhaTela() {
  const { usuario, temPermissao } = useAuth();
  const { mes } = useApuracao();
  const [dados, setDados] = useState([]);
  const [carregando, setCarregando] = useState(true);

  useEffect(() => { carregar(); }, [mes]);

  async function carregar() {
    setCarregando(true);
    try {
      const { data } = await api.get('/recurso', { params: { mes } });
      setDados(data);
    } finally { setCarregando(false); }
  }
  // ...
}
```

Sem React Query, SWR ou cache global: cada tela busca o que precisa e guarda em estado local.

Modais — o componente `Modal.jsx`

Antes, cada tela montava o seu próprio modal com `onClick={fechar}` no overlay. Um clique acidental fora da janela **apagava todo o preenchimento**. O componente padrão resolve isso:

- **não fecha ao clicar fora** — a saída é sempre consciente;
- cabeçalho estilo Windows: 💾 salvar (verde) e ✕ fechar (vermelho);

- rodapé opcional com botões nomeados.

```
<Modal titulo="Nova receita" onFechar={() => setAberto(false)} onSalvar={salvar} salvando={salvando}>
  <input ... />
</Modal>
```

> **Ao criar qualquer modal novo, use este componente.** Já foi aplicado em Financeiro/Receita, Agenda e Prestadores; os demais tiveram o `onClick` do overlay removido.

PWA e notificações

- `sw-push.js` — Service Worker com `NavigationRoute` e recepção de push.
- `main.jsx` — checagem periódica de nova versão (evita app preso em build antigo).
- `utils/notificacoes.js` — pede permissão e envia a assinatura ao backend.
- Nome do app instalado: **"Cerqueira Eng."**

> **iOS:** notificações push só funcionam se o site estiver **instalado na tela de início** (iOS 16.4+). Limitação da Apple, não do sistema.

Estilo

CSS global em `index.css` com classes utilitárias (`.btn-success`, `.btn-secondary`, `.btn-sm`, `.card`, `.modal-overlay`, `.modal-content`, `.layout-*`), complementado por estilos inline para ajustes pontuais. Responsividade por media queries: em telas pequenas o menu lateral vira gaveta com barra superior.

---

10. Domínios funcionais

10.1 Obras e o sistema de células

Geração das chaves — `client/src/utils/celulasPredio.js` é a fonte da verdade no frontend; `server/utils/celulasLabel.js` faz o caminho inverso (chave → rótulo) no backend. **Os dois precisam concordar.**

A lista de células é gerada de cima para baixo, como no desenho:

```
Cobertura → Caixa d'água → Ático → andares (topo ao térreo) →
Transição → Térreo → Laje Piso Térreo → Fundação → Itens do térreo
```

Formato das chaves:

Padrão	Exemplo	Significado
`apto-b{bloco}-a{andar}-{pos}`	`apto-b0-a3-2`	3º apto do 4º andar, bloco 1
`corredor-b{b}-a{a}`	`corredor-b0-a3`	Corredor daquele andar
`elevador-b{b}-a{a}` / `escada-b{b}-a{a}`		Idem
`pav-b{b}-a{a}`	`pav-b0-a3`	Pavimento inteiro (modo `pavimento`)
`pav-b{b}-a{a}-frente` / `-fundo`		Modo `frente_fundo`
`terreo-{i}`, `terreo-grupo`		Térreo
`fundacao-{n}`	`fundacao-2`	Etapa de fundação
`cobertura-{item}`, `terreoitem-{item}`		Itens livres
`caixa_dagua`, `atico`, `viga-transicao`, `pilar-{i}`		Fixos

> Índices de bloco e andar são **0-based**; a posição do apartamento também. Mas o número exibido do andar é 1-based e sequencial entre blocos.

Numeração automática dos apartamentos — o multiplicador se adapta à quantidade por andar: até 9 aptos/andar → ×10 (11, 12… 101, 102); 10 a 99 → ×100 (101… 1001). O térreo nunca recebe prefixo. Rótulos podem ser sobrescritos em `obra_apto_rotulos`.

> ⚠️ **Mudar a geometria da obra depois de ter marcações é perigoso.** Remover um andar ou alterar a quantidade de apartamentos muda as chaves geradas, e marcações antigas podem ficar órfãs (apontando para chaves que não existem mais).

10.2 Medição — como o valor é calculado

Esta é a regra mais importante e a que mais gerou defeitos. **Três fontes concorrem**, nesta ordem de prioridade:

```
1. Total digitado manualmente  →  vence tudo (valor fechado combinado)
2. Qtd × Unitário da pessoa    →  colaborador_precos (preço específico)
3. Qtd × Unitário do serviço   →  obra_servicos.valor_unitario (padrão)
```

```
function calcularTotalPessoa(id) {
  const totalManual = campo(id, 'total');
  if (totalManual !== '') return Number(totalManual) || 0;   // prioridade 1
  return calcularPorQtdUnit(id);                              // prioridades 2 e 3
}
```

> 🔴 **Defeito histórico (corrigido):** ao abrir uma medição para edição, o sistema pré-preenchia o campo **Total**. Como o total manual tem prioridade, alterar a quantidade mudava a metragem mas **não o valor**. Hoje a edição preenche `qtd` e — quando o unitário efetivo difere do padrão — também `unit`, deixando o total ser recalculado. Há ainda um aviso visual (borda laranja + botão ↻) quando um total fixado diverge de `qtd × unit`.

Rateio — numa execução compartilhada, o total da célula é dividido igualmente:

```
const valorTotal = qtdAtual * (servico.valor_unitario || 0);
const valorPorMembro = valorTotal / linhas.length;
const qtdPorMembro = qtdAtual / linhas.length;
```

Consolidação mensal (`/api/medicoes/gerar`): agrupa por colaborador todas as células do mês, soma, e inclui também quem só teve diária ou pagamento antecipado. Com filtro de obra ativo, diárias e antecipados só entram se a pessoa também tiver lançamento naquelas obras.

10.3 Recálculo automático

`server/utils/recalculoAutomatico.js`. Substituiu os antigos botões "Atualizar tudo" e "Recalcular valores", que dependiam de alguém lembrar de clicar.

Dispara quando: muda o preço unitário do serviço, o preço específico de um colaborador, a quantidade de uma célula ou o valor da diária de alguém.

Duas regras invioláveis:

1. **Meses** `PAGO` **nunca são alterados** — basta um envolvido ter recebido para a célula inteira

ficar protegida.

1. **Execução compartilhada rateia** quantidade e valor entre os executores.

Roda em segundo plano (`recalcularServicoEmSegundoPlano`): não atrasa a resposta ao usuário e qualquer falha vai apenas para o log, sem derrubar a operação principal.

10.4 Diárias, antecipados e desligamento

- **Diárias** — `quantidade × valor_diaria` do cadastro, com rateio opcional por obra

(`diaria_obras`).

- **Pagamentos antecipados** — sete campos de desconto; aceitam **valores negativos**

(saldo a favor da pessoa).

- **Desligamento** — `data_desligamento` registrada; a pessoa continua aparecendo nos

lançamentos por **35 dias de carência**, depois só com o checkbox "incluir desligados". O histórico de medições sempre mostra o nome, mesmo de desligados.

10.5 EPI

Estoque em `epi_itens` com `estoque_minimo`. A retirada grava a **assinatura do colaborador** (base64, capturada em canvas via `AssinaturaCanvas.jsx`) e gera termo em PDF (`utils/termoEpiPdf.js`).

Detalhe relevante: `epi_retirada_itens` **copia** descrição e CA do item no momento da retirada. Se o catálogo mudar depois, o termo assinado continua refletindo o que foi efetivamente entregue — histórico imutável.

10.6 ASO

Validade de **1 ano "inclusivo"**: o dia do exame conta como primeiro dia, então o vencimento é `data_base + 364 dias` (exame em 15/01/2025 vence em 14/01/2026).

A data-base é a última renovação em `aso_historico` ou, se nunca houve, `data_primeiro_aso`. Alerta a partir de **6 dias** do vencimento. Quem nunca fez ASO aparece primeiro na lista. **Apenas CLT (`tipo = 'CPF'`)** — PJ é excluído.

10.7 Financeiro — Receitas

Entradas com obra (texto livre, com autocomplete das obras cadastradas), serviço (lista fechada), valor bruto e o flag `com_nota`. Líquido: com nota desconta 11 %; sem nota, igual ao bruto. A importação por planilha Excel tolera acentos, maiúsculas e espaços ao casar o nome do serviço (`normalizarTexto`).

Status: `EM_ANALISE` → `CONFIRMADO` → `PAGO`.

10.8 Agenda

Compromissos com período de vários dias (`data_fim`), horário flexível (`HORA`, `DIA_INTEIRO`, `MANHA`, `TARDE`, `NOITE`), destinatários por usuário e/ou por perfil, e lembretes push configuráveis. Quem criou sempre vê o próprio compromisso, mesmo sem se marcar. O ADM tem filtros acumulativos (meus / marcados / internos) e há um quadro com os próximos 20 compromissos.

10.9 Backup

- **Automático:** diário, JSON com todas as tabelas listadas em `utils/tabelasBackup.js`,

mantendo os últimos 7 (`auto-YYYY-MM-DD.json`). Envio opcional ao Google Drive via conta de serviço.

- **Manual:** exportação e importação pela tela de Backup, com cópia de segurança gerada antes

de qualquer importação.

---

11. Processos em segundo plano

Dois agendadores sobem junto com o servidor (`server.js`), ambos em memória — **não há cron externo nem fila**. Se o container reinicia, os temporizadores recomeçam.

Backup automático

Primeira execução 1 minuto após o boot, depois a cada 24 h. Gera o JSON, tenta copiar para a pasta local do Drive (ambiente de desenvolvimento), envia pela API do Drive (produção) e limpa backups com mais de 7 dias.

Agendador de notificações (`utils/lembretes.js`)

Roda **a cada minuto**, com primeira verificação 20 s após o boot.

> 🕐 **Fuso horário — ponto crítico.** O servidor está em us-east-1. A hora local da máquina **não pode** ser usada. Todo cálculo é feito explicitamente em **UTC−3** (Brasília):
>
> ```
> const FUSO_BR = -3;
> function agoraBrasilia() {
>   return new Date(new Date().getTime() + FUSO_BR * 60 * 60 * 1000);
> }
> ```

Dois tipos de aviso:

Tipo	Quando	Para quem
Lembretes da Agenda	Conforme `minutos_antes` de cada compromisso	Destinatários do evento
ASO e Experiência	Sempre **08:00**, uma vez por dia	Perfis de alerta do RH

Regras de data dos avisos fixos:

- Fim de semana **recua para sexta** (`recuarParaDiaUtil`): vence sábado ou domingo → avisa sexta.
- Há também um aviso prévio de 24 h, igualmente ajustado para dia útil.
- Experiência avisa em **dois marcos**: 45 e 90 dias (`data_admissao + dias − 1`).
- Se o servidor estiver fora do ar às 08:00, o aviso sai assim que voltar, no mesmo dia.

Anti-duplicidade: antes de enviar, grava uma chave única em `notificacoes_enviadas`. Se a chave já existe, não envia. Para compromissos sem hora exata, usa horário de referência: `DIA_INTEIRO` / `MANHA` = 8 h, `TARDE` = 13 h, `NOITE` = 18 h.

Limpeza automática: assinaturas push que retornam 404/410 (app desinstalado, permissão revogada) são removidas do banco.

---

12. Convenções e armadilhas

Idioma do código

Tudo em português: nomes de variáveis, funções, tabelas, colunas e comentários. `calcularTotalPessoa`, `obra_servico_celulas`, `recalcularServicoEmSegundoPlano`. Mantenha o padrão — misturar inglês deixa o código inconsistente.

Comentários explicam o *porquê*

O código é fortemente comentado, e os comentários registram **a razão histórica** da decisão — geralmente um defeito real que aconteceu. Exemplo:

```
// "<> 0" (não apenas "> 0"): um total negativo é um lançamento válido também — acontece
// quando Vale Extra ou Vale Ex RH são lançados com valor negativo (saldo a favor da pessoa).
```

> **Não remova esses comentários.** Eles são a memória do projeto. Ao corrigir algo não trivial, escreva o porquê no mesmo estilo.

⚠️ Armadilhas que já causaram problemas reais

1. Encoding — nunca use `Set-Content` em arquivos com acentos Corrompe UTF-8 (vira `MediÃ§Ã£o`). Use sempre a ferramenta de edição do editor. Isto já afetou arquivos do projeto.

2. JSX — comentário logo após `&& (` quebra o build

```
{/* ERRADO — derruba o build */}
{condicao && (
  {/* comentário aqui */}
  <div>...</div>
)}

{/* CERTO */}
{condicao && (
  // comentário assim
  <div>...</div>
)}
```

3. `constraintContem()` compara nome, não conteúdo Usar essa função como verificação derrubou o servidor num loop de `process.exit(1)`. Para checar o conteúdo de uma constraint, use `pg_get_constraintdef()`.

4. Deploy "success" não significa aplicação no ar Confirme sempre via API do GitHub, SSH e requisição ao site.

5. Rota nova sem `app.use()` impede o servidor de subir É proposital (ver `verificarRotas.js`), mas surpreende quem não conhece.

6. Alterar geometria de obra com marcações existentes gera células órfãs

Checklist para adicionar um módulo novo

```
Backend
  □ server/routes/novo.js
  □ app.use('/api/novo', require('./routes/novo'))   ← obrigatório
  □ router.use(autenticar) e permissaoModulo('novo')
  □ Tabelas em database.js (CREATE TABLE IF NOT EXISTS)
  □ Adicionar 'novo' à lista MODULOS e ao PADRAO de permissões
  □ Incluir tabelas novas em utils/tabelasBackup.js   ← fácil de esquecer

Frontend
  □ client/src/pages/Novo.jsx
  □ <Route path="novo" ...> em App.jsx
  □ NavLink com temPermissao(...) em Layout.jsx
  □ Entrada em TITULOS_ROTA (título no mobile)
  □ Usar <Modal> nos formulários

Validação
  □ npm --prefix client run build (se houver Node local)
  □ Commit, push, confirmar deploy de verdade
```

Git e deploy

```
git add caminho/do/arquivo
git commit -m "Descrição objetiva do que mudou"
git push origin main
# aguardar 5-10 min e CONFIRMAR o deploy
```

Mensagens em português, no imperativo, descrevendo o efeito. Evite acentos na mensagem se o terminal tiver problemas de encoding.

---

13. Dívida técnica e melhorias

Dívida conhecida

Item	Onde	Impacto
`JWT_SECRET` fixo no código	`utils/auth.js`	🔴 Segurança — mover para `.env`
Zero testes automatizados	Projeto todo	🔴 Toda validação é manual
`ObraDetalhe.jsx` com ~2000 linhas	`pages/`	🟡 Difícil de manter
JSON como TEXT	`obras.*`	🟡 Migrar para `jsonb`
Booleanos como INTEGER	Várias tabelas	🟢 Herança do SQLite
Sem índices explícitos	Banco	🟡 Só os de PK/UNIQUE
Agendadores em memória	`lembretes.js`	🟡 Sem redundância
Vestígios do Render	`/ping`, `render.yaml`	🟢 Limpeza cosmética
Token em `localStorage`	`api.js`	🟡 Exposto a XSS

Pendência funcional em aberto

Financeiro → Receita: extrair relatório. Solicitado e ainda não implementado. Deve pedir intervalo de datas e tipo de entrada (ex.: pintura diária ou produção).

Esboço de implementação:

- Backend: `GET /api/financeiro/receitas/relatorio?inicio=&fim=&servico=` em `routes/financeiro.js`, reaproveitando os filtros existentes e o `xlsx` já em uso.
- Frontend: botão na aba Receita abrindo um `<Modal>` com os campos e disparando o download.

Melhorias sugeridas, por prioridade

Alta

1. Mover `JWT_SECRET` para variável de ambiente.
2. Testes automatizados do cálculo de medição e do rateio — é a área mais sensível e a que mais apresentou defeitos.
1. Resolver o ambiente local (Node no PATH) para permitir `npm run build` antes do push.

Média

1. Dividir `ObraDetalhe.jsx` em componentes menores.

2. Índices em `obra_servico_celulas (mes_ciclo, obra_servico_id)` e `medicoes (mes_ciclo)`.
3. Padronizar todos os modais restantes com `<Modal>`.
4. Log estruturado em vez de `console.log`.

Baixa

1. Migrar colunas JSON para `jsonb`.
2. Remover vestígios do Render.
3. Normalizar booleanos para `BOOLEAN`.

Como começar a mexer no projeto

1. Leia este documento inteiro.
2. Leia `server/db/database.js` — o esquema explica o domínio melhor que qualquer texto.
3. Leia `client/src/utils/celulasPredio.js` — entender as células é entender o sistema.
4. Leia `server/utils/recalculoAutomatico.js` — concentra as regras financeiras.
5. Só então abra as telas.

E, acima de tudo: **confirme o deploy de verdade antes de dar qualquer tarefa por concluída.**

Documento gerado a partir da leitura do código-fonte em novembro de 2026. Ao alterar a arquitetura, atualize este arquivo no mesmo commit.